



DAT400 Lab 4: OpenMP

Contacts: Hari Abram, Sonia Rani Gupta, Jing Chen, Miquel Pericas

September 23, 2025

Now that you have incrementally worked your way through a parallel implementation of the GEMM kernel using the `Pthread` library, it is important to explore the basic features of the OpenMP threading library since it represents a portable standard that is highly utilized in modern parallel applications. Don't forget to set `OMP_NUM_THREADS=number_of_processors` in order to specify the number of threads to be used by the runtime.

1 OpenMP parallel for

Parallelize the outer most loop for the GEMM implementation.

1.1 Task

- 1) Print out the thread id inside the parallel region.
- 2) Print out the number of threads only once inside the parallel region. Only one thread should be able to print, what approach you used to achieve this?

2 Parallel performance gains

This is a strong-scaling experiment that is how the solution time varies with the number of processors for a fixed total problem size. It is an implication of Amdahl's law defined in the lecture.

2.1 Task

- 1) Using `omp_get_wtime()`, time the differences between the serial and parallel implementation for sizes $P = 1, 2, 4$ threads. Report the timings.
- 2) What is the obtained parallel efficiency?

3 OpenMP scheduling policies

You have learned about static, dynamic and guided OpenMP scheduling policies. This exercise should enable you to study the differences between them using different grain sizes.

3.1 Task

- 1) For the GEMM kernel, attempt the 3 scheduling policies.
- 2) Tabulate the effect of 4 different chunk sizes for each policy (chunk size $\leq N/P$). A chunk size of 1 should be included.
- 3) What is the optimal (policy, chunk size) tuple according to your experiment and why.

4 Which loop to parallelize?

Since there is inherent data parallelism in each level down the nested loops, it is often important to know why a certain choice is preferred over the other. In this exercise, you will gain insight into the implications of the different choices.

4.1 Task

- 1) Parallelize the j (column) loop of the GEMM instead of the outer most i loop.
- 2) Parallelize the k loop. What needs to be done and why?
- 3) Compare the performance of the 3 different strategies. Explain your findings

5 Exploring SIMD

To exploit SIMD performance for data parallel loops, certain optimizations may require to be applied either manually or automatically (by the compiler). In this part, you will be checking the vector report as output by the gcc compiler, and try to trigger vector optimization. The following flags may be helpful with the gcc compiler

- To print out info about the loops that are successfully vectorized, add the `-fopt-info-loop-vec-optimized=file_name` compiler flag.
- To print out info about the loops that are not vectorized, add the `-fopt-info-loop-vec-missed=file_name` compiler flag.
- To enable vectorization, consider using `#pragma omp simd`

5.1 Task

- 1) Using the info you obtain from the compiler report with the naive GEMM implementation (with no loop permutation or tiling). Was the compiler able to vectorize the GEMM loops? Check with `-O1`, `-O2` and `-O3` optimization flags?
- 2) Now, with the loop permutation that you applied in lab 2, redo step 1. What loop was vectorized, if any, and what is the optimization flag that triggered vectorization?
- 3) How much speedup have you obtained from the vectorized vs. the non-vectorized version from step 2?

6 Tracing OpenMP programs

There are many ways to investigate the performance of parallel programs. One of the ways to achieve this is with the assistance of profiling/tracing tools. Extrae is an open-source tracing tool, which, among other things, is able to i) intercept the calls to the OpenMP runtime and provide a timeline of the activity per thread, and ii) collect performance details for each thread, using interfaces to read hardware performance counters. In this part, you will collect a trace of the OpenMP parallel code you have developed in the previous tasks, and you will visualize it using a visualization tool named Paraver. Extrae relies on a .xml file, which configures the information to be collected. We provide such a configuration file, called `extrae_omp.xml`. Before you start, unpack `Lab4_profiling.tar.xz` in your `$HOME` directory. Inside, you will find a release of Extrae, Paraver, and other tools, and a script named `SW_setup.sh`. Run `source SW_setup.sh` in your terminal to set up execution of Extrae and Paraver. To profile your code with Extrae, you will need to:

- Set the configuration file for Extrae:
`export EXTRAE_CONFIG_FILE=$HOME/Lab4_profiling/extrae_omp.xml`
- Preload the tracing library for OpenMP:
`export LD_PRELOAD=$HOME/Lab4_profiling/SW_packages/lib/libomptrace.so`
- Run your code without any modification.
- Clear the `LD_PRELOAD` variable:
`export LD_PRELOAD=`

After successful execution, Extrae will create several files. You can then run Paraver to visualize the trace, by executing `wxparaver &`. This command will open a graphical environment. To visualize the trace in Paraver:

- Load the trace (a file with a `.prv` extension).
- Visualize the trace (option "Single Timeline Window").
- Analyze the OpenMP performance with available performance counters

6.1 Task

1) Collect and visualize traces for the last versions of your OpenMP code. Zoom into the timeline, observe the execution per thread and describe it in your report. Feel free to add interesting snapshots.

7 Report Submission

- Submit the report as a group of two. Write down the group number and CIDs of each partner (i.e., who participated in this lab).
- You will get a PASS only if all the tasks are properly addressed in the report.